Smiter MPI Developers Manual

Huw Leggate

February 8, 2019

Contents

1 SMITER MPI Developers Manual **1**

1.1 Structure . . . 1

1.2 `smiter` . . . 2

1.2.1 MPI Initialisation . . . 4

1.2.2 Communicators . . . 5

1.2.3 Running `smiter` . . . 5

1.3 `smiter` components common properties . . . 6

1.4 `geoq` . . . 7

1.5 `hdsgen` . . . 7

1.6 `powcal` . . . 8

1.6.1 Standalone initialisation and domain decomposition . . . 8

1.6.2 Input and Output files . . . 9

1.7 `decomp_t structure` . . . 9

1.8 Broadcast routines . . . 10

1.9 Log files . . . 12

1.10 Other changes . . . 12

1.11 Adaptation to Chain1 . . . 13

1.11.1 Chain1 development . . . 13

1 SMITER MPI Developers Manual

1.1 Structure

The MPI changes to the smiter code consist of 3 basic additions to the existing smiter package.

- `powcal` decomposition over track starting locations
- Broadcast routines to communicate data both within `powcal` and between the `smiter` steps
- New `smiter` program unit that allows the running of the smiter package as a single executable

Most MPI related code is contained in two new files, `smitermpi_m.f90` which contains all the MPI library calls and MPI related subroutines and `smitermpi_h.f90` which contains the declaration of the `decomp_t` structure (see 1.7) and a global variable `myrank_log` which is used in limited cases to avoid adding extra arguments to existing smiter subroutines, particularly in the log files. This variable replaces `myrank_global` used in earlier versions. The smiter code uses functional and domain decomposition to allow the most efficient running of the code. The `powcal` component of smiter is decomposed over the domain of track starting positions while the `geoq` and `hdsgen` components are decomposed functionally. The new MPI code follows the style and conventions already in place in the smiter suite of codes.

The smiter functional decomposition is part of the changes made to allow smiter to run as a single executable, combining the 3 smiter components that were previously compiled as separate executables, `geoq, hdsgen` and `powcal` into a single executable. Changes were made to each of the 3 smiter component source files `geoq.f90, hdsgen.f90` and `powcal.f90`. These files originally each contained a single program unit, these have now been altered to Fortran90 modules with the suffix `_p` to differentiate them from the other module files with suffix `_m`. Each module contains a single subroutine with the suffix `_sub` and three arguments, `arg, combined` and `decomp`. A program unit with the same name as the original has also been added to each file which calls only the relevant subroutine. This program unit is contained within an `#ifndef` preprocessor directive so that by default each component will compile and link as its own executable in the same way as was done previously using the relevant Makefile. In addition a new file `smiter.f90` has been added to the repository along with its Makefile `Makefile.smiter`. This file contains a single program unit that initialises MPI and calls each of the `_sub` subroutines corresponding to the smiter components. The control flow of this program unit can be seen schematically in figure 1. A more detailed description of the `smiter` program and each of the smiter components is presented below.

1.2 `smiter`

The `smiter` program unit controls the distribution of smiter components to available processes. Here the term component will be used to describe the 3 subroutines or program units that make up the core `smiter` package, `geoq, hdsgen` and `powcal`. The term step will be used to describe the 4 specific calculations being carried out, `geoq` shadow, `geoq` results, `hdsgen` and `powcal`. `smiter` first calls the subroutine `smitermpi_init` which initialises MPI and creates groups and communicators

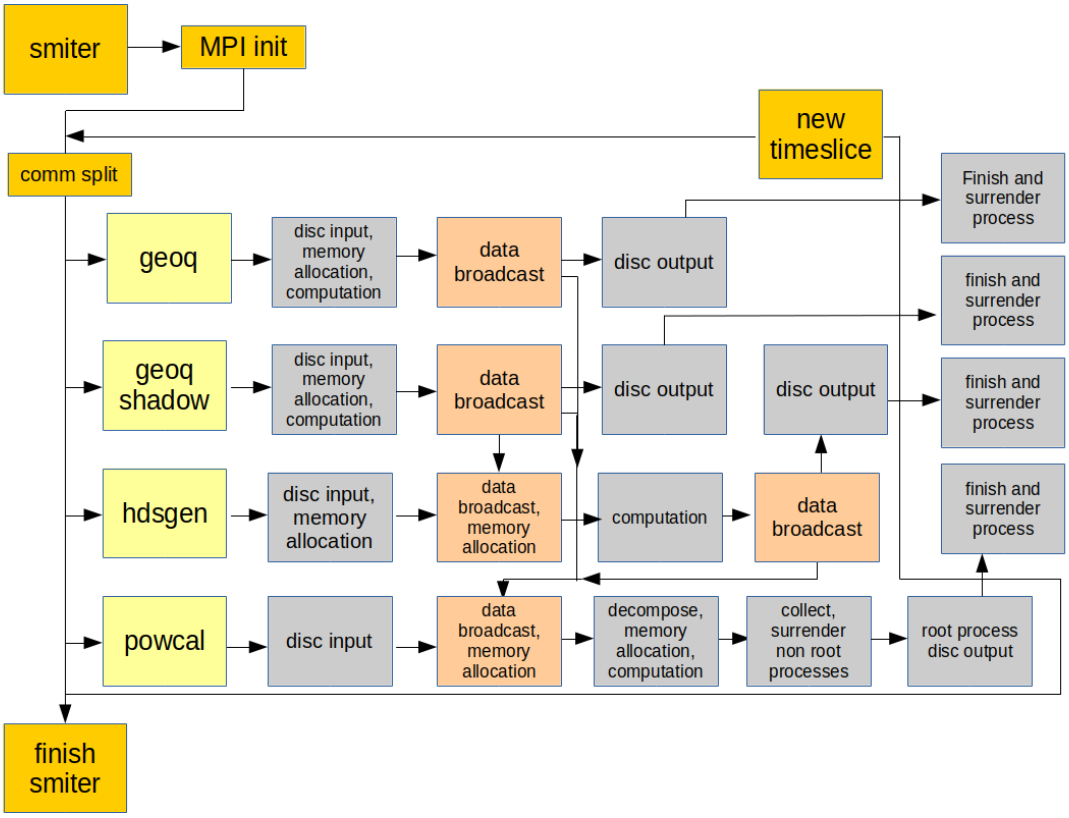


Figure 1: Control flow for the smiter when run as a single program unit. Orange refers to tasks carried out on all available processes by the smiter program unit. Yellow refers to the existing smiter program units as subroutine calls. Grey refers to calls by the subroutines. Pink indicates communication with the main smiter control flow.

for all of the components to be run. The process for each component is set during initialisation and the smiter program then instructs the correct process to run each component using the `decomp` structure, 1.7. `geoq` and `hdsgen` are run on single processes and `geoq` is run twice, once for the shadow and once for the results, different communicators are used for each of these tasks. The execution of the `powcal` step is controlled by the logical `powcal` in the `decomp` type so that any process with `powcal` set `True` will call the powcal step. Data broadcasts between steps are dealt with solely by the individual components and not by the `smiter` program unit. A schematic of the flow control can be seen in figure 1.

In summary `smiter` initially calls all 4 smiter steps which read configuration data from disk. `geoq` shadow and results and the `hdsgen` steps all run on a single process while `powcal` runs on as many processes as are available. The `geoq` shadow and `geoq` results steps then run to completion. The `hdsgen` step waits for a data broadcast from the `geoq` shadow step and itself then runs to completion. The `powcal` step then receives data broadcasts from the `geoq` shadow and results steps and then waits a short time for the creation and broadcast of the Hierarchical Data Structures from the `hdsgen` step. Once `powcal` has completed its track evaluation the non root processes are released and powcal uses only the root process to write to disk. In practice the runtime of the full set of codes is defined by the run time of the `geoq` shadow step which spends most of its runtime writing diagnostic data to disk.

1.2.1 MPI Initialisation

Smiter initialises MPI in the subroutine `smitermpi_init`. A duplicate of the `MPI_COMM_WORLD` communicator is store in the `commsmiter` variable in the `decomp` type. This calls `smitermpi_control` which decides on the runtime behaviour and sets a series of control flags governing process assignment and data broadcasts. If 6 or less processes are available smiter will run the 3 components to completion before starting powcal as the extra cores available for `powcal` improve the runtime When running on 2 cores the `geoq` step and `hdsgen` step are run consecutively while `geoq` results is run concurrently with these. When running on 3 or more processes each step is run concurrently and data is broadcast between `geoq` shadow and `hdsgen`. In the case where 7 or more processes are available `powcal` starts as soon as data is available from the `geoq` and `hdsgen` components. `smitermpi_control` then calls `smitermpi_decomp` which create groups and communicators for each of the steps, see 1.2.2 for a description of these communicators.

Communicator	Members	Purpose
`commsmiter`	All processes	Copy of `MPI_COMM_WORLD`
`commpowcal`	Processes running `powcal`	Control of powcal decomposition and broadcasts within powcal
`commgeoqshad`	Union of `geoq` shadow and powcal processes	Broadcasts from `geoq` shadow to `powcal`
`commgeoqres`	Union of `geoq` results and `powcal` processes	Broadcasts from `geoq` results to `powcal`
`commhdsgen`	Union of `hdsgen` and `powcal` processes	Broadcasts from `hdsgen` to `powcal`
`commshadhds`	Union of `geoq` shadow and `hdsgen` processes	Broadcasts from `geoq` shadow to `hdsgen`

Table 1: Communicators stored in the `decomp` type used for smiter control.

`powcal` domain decomposition is not carried out in `smiter` but instead in the `powcal` component using the subroutine `powcalmpi_decomp`.

1.2.2 Communicators

At initialisation smiter creates a set of groups and communicators for each individual step and for communication between steps. This is carried out using the MPI routines `MPI_Group_incl`,`MPI_Group_excl` and `MPI_Group_union`. Table 1 shows the members and purpose of each communicator when running as a single executable. When powcal is running on all processes or as a standalone executable all communicators are set to `commpowcal`.

1.2.3 Running `smiter`

The single executable version of smiter now takes 4 input files on the command line, one for each step. The `smiter` program has not been included in the existing smiter execution scripts and so in order in run the directory structure usually created by these scripts must exist. This consists of 4 directories, `S, G, H, P`, each containing the relevant `.ctl` control file used in the original smiter

code. To run with MPI the new `smiter` executable is run from the directory above this with the command

```
> mpiexec -np 4 smiter S/<shadow>.ctl G/<results>.ctl H/<hdsgen>.ctl P/<powcal>.ctl
```

Using the appropriate configuration files. As smiter runs in the top directory a small change is needed to the input files in that the paths in the `&inputfiles` namelist need to have the `'../'` removed, e.g. vtk_input_file='../S/jetbean_geoqm.vtk' becomes vtk_input_file='S/jetbean_geoqm.vtk'. The geometry and equilibrium input files then need to be added to the top directory or the paths in the `.ctl` input files changed to include the releveant directory, e.g. `vtk_input_file='jet_divertor.vtk'` becomes `G/vtk_input_file='jet_divertor.vtk'`.

1.3 `smiter` components common properties

There are several common changes to the three smiter components. Each component now consists of a module, `geoq_p, hdsgen_p, powcal_p` and a program unit with the same name as the original code, `geoq, hdsgen, powcal`. The program unit is within a preprocessor directive `#ifndef COMBINED` so that when the original Makefile is used the the program unit is seen. However when the preprocessor flag COMBINED is set these program units are not visible by the compiler and only the module within the file is compiled. The `_p` modules contain a single subroutine with the suffix _sub i.e. `geoq_sub, hdsgen_sub, powcal_sub`.

The component subroutines take three arguments, `integer arg, logical combined` and `decomp_t decomp`, see 1.7 for details of the decomp_t derived type. The integer `arg` is used as input to the `get_command_arg` subroutine call and sets the number of the command line argument to be read at runtime. The logical `combined` notifies the subroutine that it is being run as part of the single `smiter` executable and enables broadcasts to other components as well as other small changes to the runtime behaviour of each code. The `decomp` argument contains the MPI decomposition information that is initialised in the `smiter` program unit. When the `powcal` component is run as a standalone executable the `decomp` type is initialised by powcal program unit instead of smiter. When `geoq` and `hdsgen` are rune standalone the decomp type is declared in the program unit but is not initialised as these code only run on a single core.

1.4 geoq

Apart from the common changes described previously there is only a further single line change to the geoq.f90 file. If the input argument combined is True a call is made to a new routine geoq_bcast which is located in the file geoq_m.f90. This routine broadcasts relevant data to the powcal and hdsgen steps using bcast routines described in 1.8. geoq_bcast operates differently depending whether geoq is generating the results or shadow geometry.

When generating results geometry, broadcasts are made to the powcal step through a call to bcast_powcal followed by a call to bcast_position both using the commgeoqres communicator. The call to bcast_position is due to the bcast_powcal routine broadcasting the position data of the geobjlist argument. When the powcal component reads from disk the stored position data has been transformed into R, Z, Φ coordinates but is not stored in this way in the geoq data types. The subroutine geoq_makefrzxi creates a position type with this transform carried out. It is also necessary to transform position data into B-field coordinates for the call to bcast_powcal, this is done using the routine geoq_makebcart before the call to bcast_powcal.

When generating shadow geometry broadcasts are made to hdsgen and powcal. The broadcast to hdsgen is carried out first with a call to bcast_geobjlist using the commshadhds communicator. It is again necessary to rebroadcast position data with data transformed using geoq_makefrzxi. Data is the broadcast to the powcal step. A call to bcast_beq is made followed by a call to bcast_skyl if skylights are being used. A call is then made to bcast_geobjlist with a further rebroadcast of position.

1.5 hdsgen

When running in combined mode The hdsgen step both receives broadcast data and acts as the root for broadcasts. When running as a standalone program hdsgen reads data from the geoq shadow vtk file, in combined mode this is replaced with a call to bcast_geobjlist from geoq shadow using the comshadhds communicator. A call to bcast_position is also required to account for transformed data, see 1.4. A new argument optional logical noread has been added to the geobjlist_init subroutine that returns the routine after some initialisation but before data is read from disk. Once the Hierarchical Data Structure has been created another broadcast is made to powcal using a call to bcast_hds using the commhdsgen communicator, see 1.8 for further details.

Subroutine name	Communicator	Purpose
bcast_pcontrol	commpowcal	Control data read from file
bcast_beq	commgeoqshad	Equilibrium data read from file or geoq
bcast_skyl	commgeoqshad	Skylight data read from file or geoq shadow
bcast_geobjlist	commgeoqshad	Geometry data read from file or from geoq
bcast_position	commgeoqshad	Update for transformed position data
bcast_powcal	commgeoqres	Geometry data read from file or geoq results
bcast_position	commgeoqres	Update for transformed position data
bcast_hds	commhdsgen	HDS data read from file or hdsgen

Table 2: Broadcasts used in the powcal routine.

1.6 powcal

Several changes have been made to the powcal component of smiter. Broadcasts are made when running in combined mode and also when running in standalone parallel mode. The subroutines called to carry out these broadcasts are presented in the list below. Further details of these routines can be seen in section 1.8. When data is to be read powcal checks whether that data is to be read from file or a broadcast received from another step. If the data is read from file and more than one process is being used the data is broadcast from powcal process 0.

1.6.1 Standalone initialisation and domain decomposition

MPI initialisation for the standalone powcal is carried out in powcalmpi_init which takes a single argument of type decomp_t. A duplicate of MPI_COMM_WORLD is created and stored in the commpowcal member of the decomp_t structure to simplify the extension of MPI beyond just the powcal program.

As the smiter code calculates a series of tracks along field lines the decomposition is performed over the starting locations of these tracks. The spatial domain itself is not decomposed as each track can potentially pass through any part of the domain. This allows a very simple decomposition over track number which is carried out in the subroutine powcalmpi_decomp for both the standalone and combined configurations. The tracks are distributed evenly using the routine powcalmpi_localsize. Information on the decomposition is stored in a derived type decomp_t

decomp, which is defined in powcal_m.f90m for the standalone configuration, see 1.7.

1.6.2 Input and Output files

Input from file has been restricted to the root process 0. Data is then broadcast from this process to all other processes using the broadcast routines detailed in section 1.8.

Output to disk is carried out by the root process 0 once all processes have completed track evaluation and the data has been gathered by the root process. The gather step is carried out by the powcalmpi_collect(powcal_t powcal) routine. This calls MPI_Gatherv using MPI_IN_PLACE for the arrays pows, powa, pow, psista, and pmask from the powcal%powres structure. The complete data is then output in the same manner as for the serial code.

1.7 decomp_t structure

The decomp_t type declaration is contained in the file smitermpi_h.f90. This type contains all the information related to the MPI operation of smiter and its components. Although it is possible to obtain much of this information from the MPI API this approach allows values to be assigned when running in the absence of the MPI library and also simplifies syntax in a number of situations.

```
type, public :: decomp_t
   integer(ki4) :: myrank = 0      ! Global Rank of this process
   integer(ki4) :: myrankgeoqshad = 0     ! Rank of this process in the geoqshad group
   integer(ki4) :: myrankgeoqres = 0     ! Rank of this process in the geoqres group
   integer(ki4) :: myrankhdsgen = 0     ! Rank of this process in the hdsgen group
   integer(ki4) :: myrankpowcal = 0     ! Rank of this process in the powcal group
   integer(ki4) :: myrankshadhds = 0     ! Rank of this process in the shadhds group
   integer(ki4) :: nproc = 1       ! Number of processes
   integer(ki4) :: nprocpowcal = 1      ! Number of processes in powcal group
   integer(ki4) :: nprocgeoqshad = 1      ! Number of processes in geoqshad powcal group
   integer(ki4) :: nprocgeoqres = 1      ! Number of processes in geoq res powcal group
   integer(ki4) :: nprochdsgen = 1      ! Number of processes in hdsgen powcal group
   integer(ki4) :: nprocshadhds = 1      ! Number of processes in shadow hdsgen group
   integer(ki4) :: firstpowe,lastpowe ! First and last track numbers
   integer(ki4) :: firstng,lastng  ! First and geometry indices
```

```
    integer(ki4) :: mynpowe,myng    ! The local number of tracks and tracks x levels
    ! Arrays holding the size and displacements of tracks for each process
    integer(ki4),dimension(:),allocatable :: localsize, localdisp
    integer(ki4) :: commsmiter      ! smiter communicator, currently=MPI_COMM_WORLD
    integer(ki4) :: commpowcal      ! powcal communicator
    integer(ki4) :: commgeoqshad       ! geoqshad and powcal communicator
    integer(ki4) :: commgeoqres      ! geoqresults and powcal communicator
    integer(ki4) :: commhdsgen      ! hdsgen and powcal communicator
    integer(ki4) :: commshadhds      ! geoq shadow and hdsgen communicator
    integer(ki4) :: procgeoqshad = 0   ! The process assigned to run geoq shadow
    integer(ki4) :: procgeoqres = 0  ! The process assigned to run geoq results
    integer(ki4) :: prochdsgen = 0  ! The process assigned to run hdsgen
    logical :: powcal = .False. ! If true this process runs the powcal step
    logical :: bcasthdsgen = .False., bcastpowcal = .False. ! If true then hdsgen and powcal
                                                            ! expect data as broadcasts
end type decomp_t
```

The `firstpowe` and `lastpowe` variables define the range of tracks calculated by each process. The `localsize` and `localdisp` arrays are required for the collection of power deposition data after all tracks have been calculated.

`myrank_log` As well as the declaration of the `decomp_t` type `smitermpi_h.f90` also contains a global variable `myrank_log`. This is set to the root process for each component and allows the suppression of log messages from non root processes without the need to pass subroutine arguments in the log calls as this would require changes in a large number of files. The use of this variable is kept as low as possible.

1.8 Broadcast routines

The broadcast routines were first written to enable powcal to broadcast data to be read from disk by process 0 and broadcast to the other processes. In the single executable version of smiter the broadcasts take place between different steps. To enable this the process rank and communicator to be used is passed to each routine, so that the broadcast routines have the following form.

```
bcast_data( derived_type data, integer rank, integer comm)
```

The communicator used can therefore be the `powcal` communicator `commpowcal` that contains only processes that are running `powcal` or one of the other communicators that contain a union of processes running different steps. The following list presents the calls made by the higher level broadcast routines

- `bcast_pcontrol(file,numerics,onumerics,edgprof,plot,rank,comm)`
  - `bcast_pnumerics(numerics,rank,comm)`
    - `bcast_termplanes(termp,rank,comm)`
  - `bcast_edgprof(edgprof,rank,comm)`
  - `bcast_odes(onumerics,rank,comm)`
  - `bcast_file(numerics,rank,comm)`
  - `bcast_pplot(plot,rank,comm)`
- `bcast_beq(beq,rank,comm)`
  - `bcast_bnumerics(numerics,rank,comm)`
  - `bcast_spl2d(numerics,rank,comm)`
- bcast_powcal(geobjlist,vecx,vecb,rank,comm)
  - `bcast_geobjlist(geobjl,rank,comm)`
    - `bcast_position(posl,rank,comm)`
  - `bcast_position(vecx,rank,comm)`
  - `bcast_position(vecb,rank,comm)`
- `bcast_geobjlist(gshadl,rank,comm)`
  - `bcast_position(posl,rank,comm)`
- `bcast_hds(gshadl,gnumerics,btree,rank,comm)`
  - `bcast_numerics(gnumerics,gshadl,rank,comm)`

  - bcast_btree(btree,rank,comm)
    * bcast_ls(objectls,rank,comm)
- bcast_skyl(skyl,rank,comm)
  - bcast_sknumerics(n,rank,comm)

The top level items are called from either the powcal program unit after each relevant read or from the appropriate step when running as a single executable. Subsequent levels in this list refer to calls made by the higher level subroutine. Each of these broadcast routines also allocates memory and calls initialisation routines where required. The ordering of these broadcasts is based on the ordering of serial file input. It may be possible to improve the efficiency of this process by reordering the calls however this has not been attempted.

1.9 Log files

Smiter periodically writes error logs to stdout during track evaluation. Each component writes to its own log file. When run in parallel mode this causes a large overhead as multiple processes attempt to write to stdout. To avoid this error messages are added to a new new error buffer which can be flushed when required. This buffer has been added to log_h.f90 and the new file log_h.f90. The buffer is of type error_buffer_t containing error messages in an array of type error_line_t. The buffer is allocated in log_init to a default size of ebufinitsize lines, currently set to 32. The buffer grows when full so as to limit the memory used for long runs and is set to a maximum size of 100000 lines. The buffer is flushed with a call to log_flusherror after the completion of track evaluation. Code to gather log file from non-root processes was not completed during the project.

1.10 Other changes

The pre-existing timing routines in Smiter have been used to add timing of the MPI portions of the code including a check for imbalance after initialisation. Two new config files have been added to the config directory for the building of the Smiter executables. config_gfortranmpi.cfg adds support for compilation using the gfortran compiler and MPI libraries and config_intelmpi.cfg adds support for compilation using the Intel Fortran compiler and MPI libraries.

A new makefile `Makefile.smiter` has been added to the `f95` directory in the smiter repository that builds the single executable version of the code. This has not been incorporated into the existing smiter build scripts.

1.11 Adaptation to Chain1

The single executable of `smiter` has been written with the potential to include the code in the JET intershot suite Chain1. The expectation in Chain1 is that `smiter` would run a set number of time slices. Although this would be theoretically possible with the existing smiter code running repeatedly the overheads incurred would make this unfeasible. The single executable version could be adapted to run multipple time slices efficiently with a time spent per time slice in the region of 10 seconds when running on approximately 100 processes.

Multiple time slices can be processed in the following manner. Data for the first timeslice is read, the `geoq` and `hdsgen` steps run and pass data to the `powcal` step running on three fewer than the total number of available cores. Once the `powcal` step has completed track evaluation all processes other than rootare then available for the next time slice. The 4 processes running `geoq, hdsgen` and `powcal` continue writing data to disk. At this point a new decomposition is created based on the now finished `powcal` processes, now 4 fedwer than the original number of processes. The process repeats so that on the 3rd iteration there will be 8 fewer than the original number of processes. This continues with smiter running on gradually fewer processes. After some time the first `geoq, hdsgen` and `powcal` root processes will complete and will then be added to the pool of processes available for each successive timestep, arresting the reduction in available processes. This is expected to take between 5 and 10 iterations depending on the configuration used.

1.11.1 Chain1 development

Significant further development would be required to include smiter in Chain1, the following lists indicates some of the expected tasks.

- Define and implement naming convention for input and output files
- Add `commall` and `commavailable` groups and communicators to allow repeat decompositions
- Develop code to monitor available processes and perform new decompositions

Several potential optimisations would be possible to improve the efficiency of the full Chain1 `smiter` code.

- Retain the CAD geometry in memory, this only needs to be read once per run not for each timeslice
- Retain initial configuration read from file in memory
- Avoid the allocation and deallocation of memory for each step when not required, mainly applicable for the `powcal` step